

The Robot Simulator, from the top down

MTRX3760 Lab 2 - draft walkthrough

August 31, 2026

CONTENTS

The Robot Simulator, from the top down	
MTRX3760 Lab 2 — how the draft works, and what to change	
1. What the program does	
What you see on screen	
Measured behaviour	
2. The coordinate system — read this before anything else	
3. The top: main() and CSimulator	
4. One update, traced all the way down	
5. CRobot — where the design decision lives	
What CRobot holds	
The kinematics: AdvancePose	
Collision counting	
Lap detection	
6. The sensors	
The shared part: CSensor	
CRangeSensor	
CLineSensor	
7. The control laws	

Wall following: two rules

The tuning numbers

Line following: three cases

8. CWorld — the room and the line

9. CGeometry — the maths at the bottom

Ray against segment

Point to segment

The rest

10. Design decisions, and what you might change

1. Inheritance for the robots

2. CWheel exists as a class

3. Sensors share a base for geometry, not for polymorphism

4. The Color leak

5. Fixed-timestep integration

6. Lap detection by distance from the start

7. Things not yet done that the spec allows

11. What is still yours

Immediate blockers

The report deliverables the code feeds

The obvious split

12. Building and running

The Robot Simulator, from the top down

MTRX3760 Lab 2 — how the draft works, and what to change

This document explains the first-draft simulator one layer at a time, starting at `main()` and descending to the trigonometry at the bottom. It is written to be read in order: each section only uses ideas from the sections above it.

The last two sections are different in kind. **Design decisions** lists the calls the draft makes and the alternatives you could take instead, and **What is still yours** lists the parts of the lab the code does not touch. Those are the sections to argue with.

1. What the program does

Two robots share one room.

The **wall follower** starts near the top left and drives clockwise around the room, keeping the wall on its right, until it arrives back where it began. The **line follower** starts on a line painted on the floor and follows that line around its own loop. They ignore each other completely — they do not collide, avoid, or even know the other exists.

Both are drawn as circles with a line showing which way they face, and both leave a trail that stays on screen for the whole run. When both have completed a lap the simulation stops, prints a summary, and leaves the window open so the finished trails can be looked at or screenshotted.

What you see on screen

Thing	Appearance
Room walls	light grey outline
Floor line	dimmer grey, thicker, inside the room
Wall follower	blue circle, blue trail
Line follower	amber circle, amber trail
Heading	white line out of the front of each robot
Range sensors	faint grey beams, red dot where they hit a wall
Line sensors	dots at the sensor, red when over the line

The sensor drawing is not decoration. It is the fastest way to tell a steering bug from a sensing bug: if the beams are pointing somewhere sensible and the robot still turns the wrong way, the fault is in the controller.

Measured behaviour

Verified by running the simulation logic without a window:

Updates to complete both laps	1624 (about 27 seconds of simulated time)
Collisions	0 (the hurdle allows up to 10)
Closest the wall follower gets to a wall	29.3 units, against a body radius of 15
Wall follower's path	spans x[150,665] y[129,461] in a room of x[100,700] y[100,500]

That last row matters more than it looks. A “completed lap” flag can be fooled by a robot that wanders off and comes back, so the path was checked against the room’s own extent and against all four quadrants. It genuinely goes round.

2. The coordinate system — read this before anything else

Everything below depends on one convention, set by the supplied `CLoopReader.h`:

- **x** runs left to right.
- **y** runs **top to bottom**, so y increases *downwards*.
- **Headings** are in radians, measured **clockwise** from the positive x axis.

So heading 0 faces right, $\pi/2$ faces **down** the screen, π faces left.

The consequence that matters everywhere: **adding to a heading turns the robot to its own right**. Picture standing on the screen facing right (east); your right hand points down the screen (south), which is exactly where heading + 90 degrees points.

Getting this backwards is the single most likely bug when you modify the code. Every turn calculation in the draft is written on the assumption that a positive turn means “to the robot’s right”, and several comments say so out loud for exactly that reason.

3. The top: `main()` and `CSimulator`

`main()` is deliberately almost empty:

```
CSimulator Simulator;

if( Simulator.Build( WallsMapFile, LineMapFile ) )
{
    Simulator.Run();
    Result = 0;
}
```

That is the whole program. There is nothing to assemble by hand because `CSimulator` owns everything: the window, the world, and every robot in it.

`Build()` loads the two map files and, if both read successfully, creates one robot of each kind, each starting from the pose named in its own map file. It returns false on a failed load so `main()` can report the problem instead of running an empty simulation.

`Run()` is the loop:

```
while( !mRender.WindowShouldClose() )
{
    if( mIsRunning )
    {
        UpdateRobots();
        ++mUpdateCount;

        if( AllRobotsFinished() || ( mUpdateCount >= MaximumUpdates ) )
        {
            mIsRunning = false;
            ReportSummary();
        }
    }

    DrawFrame();
}
```

Three things are worth noticing here.

Updating and drawing are separate. `DrawFrame()` runs every pass, but `UpdateRobots()` only runs while `mIsRunning`. That is what lets the window stay open showing the completed trails after the simulation has stopped, which is how you take the screenshot the report asks for.

The time step is fixed. Every update advances the simulation by `FixedTimeStep`, one sixtieth of a second of *simulated* time, regardless of how long the frame actually took. The handout requires this, and the reason is repeatability: the same program produces the same trajectories on a fast machine and a slow one. If you drove it from real elapsed time instead, your collision count would change depending on what else the laptop was doing, and no result would be worth reporting.

There is a maximum. `MaximumUpdates` stops a stuck robot from hanging the program forever. It is a safety net, not part of the design.

4. One update, traced all the way down

This is the spine of the whole program. Everything else is detail hanging off it. Follow one update from the top:

```

CSimulator::Run()
└─ UpdateRobots()
   └─ for each robot: CRobot::Update( world, dt )
      └─ 1. SenseAndSteer( world )      <- virtual, differs per robot
         └─ sensor.Sense( world, pose )
            └─ world.DistanceToWallAlong / IsOverLine
               └─ CGeometry::RayHitsSegment / DistanceToSegment
                  └─ ChooseTurn()        <- the control law
                     └─ Drive( leftSpeed, rightSpeed )
      └─ 2. AdvancePose( dt )            <- differential drive kinematics
      └─ 3. mTrail.AddPoint( position )
      └─ 4. CheckForCollision( world )
      └─ 5. CheckForLapCompletion()

```

The order is not arbitrary. **Steering is decided from where the robot was, then the robot moves, then the consequences of moving are recorded.** That is what a real control loop does: sense, decide, act. If you moved first and sensed afterwards you would be steering on stale information by one time step.

Notice that steps 2 to 5 are identical for both robots. Only step 1 differs, and that is precisely why step 1 is the virtual function.

5. `CRobot` — where the design decision lives

This is the class to look at first when someone asks you to justify the design.

Read the handout's two robot specifications side by side. Both robots are a disc of radius 15, drawn as a circle with a heading indicator. Both are driven by two independently controlled wheels. Both advance by a fixed slice of simulated time each update. Both leave a trail that stays visible. Both run until they have been all the way round.

The **only** difference is that one carries two range sensors and the other carries two line sensors, and consequently they decide how to steer differently.

So `CRobot` holds every one of the shared things, and leaves exactly two questions abstract:

```
// Read the sensors this robot carries and set the two wheel speeds.
virtual void SenseAndSteer( const CWorld& arWorld ) = 0;
// Draw whatever sensors this robot carries.
virtual void DrawSensors( CRender& arRender ) const = 0;
```

A derived robot never touches the kinematics, the trail, collision counting or lap detection, because none of those differ.

This is the answer to A3. The post-mortem asks *"what's an important aspect of the design of A1 that you had to change in order to make A2 work?"* With this structure the honest answer is "nothing" — A2 was a new subclass plus a new sensor type, and no existing file needed editing. Whether that is the answer you *want* to give is a separate question, discussed in section 10.

What `CRobot` holds

Member	Purpose
<code>mPose</code>	position and heading
<code>mLeftWheel</code> , <code>mRightWheel</code>	the two independently driven wheels
<code>mTrail</code>	the path drawn behind it
<code>mStartPosition</code> , <code>mHasLeftStart</code> , <code>mLapComplete</code>	lap detection
<code>mCollisionCount</code> , <code>mWasColliding</code>	collision counting

The kinematics: `AdvancePose`

A differential drive robot has two wheels it can spin at different speeds, and that is its entire means of moving. Two numbers in, a position and a heading out.

```
forward speed = (left + right) / 2
turn rate     = (left - right) / (distance between the wheels)
```

The **forward speed** being the average is easy to see: if both wheels run at the same speed the robot goes straight at that speed.

The **turn rate** is the interesting one. Drive the left wheel faster than the right and the robot swings to its right, which under our clockwise convention means the heading *increases* — hence `left - right` rather than `right - left`. Dividing by the wheel separation is what makes it an angular rate: the same difference in wheel speeds turns a narrow robot faster than a wide one.

In the code the separation is asked of the wheels rather than assumed:

```
const float WheelSeparation = mRightWheel.GetLateralOffset()
                             - mLeftWheel.GetLateralOffset();
```

Then integrate over the time step:

```
mPose.mHeading = NormaliseAngle( mPose.mHeading + TurnRate * aDeltaTime );
mPose.mPosition = PointAlong( mPose.mPosition, mPose.mHeading,
                              ForwardSpeed * aDeltaTime );
```

This is the simplest possible integration — turn first, then move in the new direction, in a straight line. Over a sixtieth of a second the error against a proper arc is far too small to see. It is worth knowing it is an approximation, because that is the kind of thing worth a sentence in a report.

Collision counting

The robot is a disc, so it is touching a wall when its **centre is within one body radius** of that wall:

```
const bool Touching = ( arWorld.DistanceToNearestWall( mPose.mPosition )
                      < BodyRadius );

if( Touching && !mWasColliding ) { ++mCollisionCount; /* report it */ }
mWasColliding = Touching;
```

The `mWasColliding` flag is the important part. Without it, a robot scraping along a wall for half a second would report thirty collisions instead of one. Counting the *transition* into contact rather than the state of being in contact is what makes the number mean “how many times did it hit something”.

Lap detection

```
if( FromStart > LapDepartureDistance )           mHasLeftStart = true;
else if( mHasLeftStart && FromStart < LapReturnDistance ) mLapComplete = true;
```

Two stages, and the first stage is what makes it work. Requiring the robot to get well away from the start before coming home can count is what stops the first few updates — when the robot is obviously still near its start — from instantly registering a lap.

It is a deliberately simple test, and you should know its limitation: a robot that drove out and straight back would satisfy it without going round. That is why the draft was verified against the room’s extent and quadrant coverage separately, rather than trusting this flag.

6. The sensors

The shared part: `CSensor`

Both kinds of sensor are bolted to the robot at a fixed place and pointing in a fixed direction *relative to the robot*. When the robot moves, the sensor moves with it. That arithmetic is the same for both, so it lives in the base class:

```
float CSensor::HeadingFor( const CPose& arRobotPose ) const
{
    return NormaliseAngle( arRobotPose.mHeading + mMountAngle );
}

Vec2D CSensor::PositionFor( const CPose& arRobotPose ) const
{
    return PointAlong( arRobotPose.mPosition, HeadingFor( arRobotPose ),
                      mMountDistance );
}
```

A sensor mounted at +90 degrees always looks to the robot's right, whichever way the robot happens to be facing.

Note what is **not** in the base class: the reading. A range sensor reports a distance, a line sensor reports yes or no. Those are genuinely different things, so each derived class exposes its own. This is discussed further in section 10 — it is one of the design calls worth revisiting.

`CRangeSensor`

Reports the distance to the first wall along the direction it faces, capped at `MaxRange`:

```
const float Raw = arWorld.DistanceToWallAlong( PositionFor( arRobotPose ),
                                              HeadingFor( arRobotPose ) );
mDistance = ( Raw > MaxRange ) ? MaxRange : Raw;
```

The cap is not only realism. Without it, the instant the robot passes an outside corner and its side sensor sees nothing at all, the reading would jump to a huge number and any control law using it would react violently. Capping turns “nothing there” into a large but finite reading, which the controller can handle sensibly.

CLineSensor

Reports a single bit: is the floor directly beneath this sensor painted line, or not.

```
mOverLine = arWorld.IsOverLine( PositionFor( arRobotPose ) );
```

That is all the line follower gets to work with — two bits per update — which is what makes its control law interesting.

7. The control laws

Wall following: two rules

The wall follower carries two range sensors, both looking right: one at **90 degrees** (directly right) and one at **45 degrees** (forward and right). Each has a distinct job.

- The **side sensor** answers *"am I the right distance from the wall?"*. This is what holds the robot parallel to a straight wall.
- The **diagonal sensor** answers *"is a wall about to be in front of me?"*. This is what gets the robot around an **inside** corner.

Why two sensors are genuinely necessary: `SimpleWalls.map` is a rectangle with a notch cut into the top edge, so it has an inside corner. Approaching one, the side sensor reads a perfect distance right up until the robot drives into the wall ahead of it — because the wall ahead is not in the side sensor's view at all. A robot with only a side sensor cannot finish this lap. The map was chosen to force this.

```
if( mDiagonalSensor.GetDistance() < DiagonalClearance )
{
    Result = -MaximumTurn;           // wall closing in ahead-right: turn left
}
else
{
    const float Error = mSideSensor.GetDistance() - TargetWallDistance;
    Result = ProportionalGain * Error; // clamped to +/- MaximumTurn
}
```

The diagonal rule has priority, and it must: at an inside corner the side reading is still good, so if the side rule won the robot would drive straight into the wall while being told everything was fine.

The second rule is **proportional control**. The error is how far the measured distance has drifted from the distance we want to hold. Too far from the wall gives a positive error and turns right, towards it. Too close gives a negative error and turns away. The gain sets how hard it reacts.

The clamp does more work than it looks. At an **outside** corner the side sensor runs off the end of the wall and reads its maximum range, giving an enormous positive error. Clamping turns that into a steady hard right turn — which is exactly the behaviour needed to round an outside corner. One clamp handles a case that would otherwise need its own rule.

Finally the turn becomes two wheel speeds:

```
Drive( CruiseSpeed + Turn, CruiseSpeed - Turn );
```

Adding to the left wheel and subtracting from the right turns the robot right, as section 5 explained, while leaving the average forward speed unchanged.

THE TUNING NUMBERS

Constant	Value	What it does
TargetWallDistance	45	how far out it tries to hold
DiagonalClearance	62	closer than this ahead-right means a corner
ProportionalGain	0.9	how hard it reacts to being off distance
MaximumTurn	42	caps the wheel-speed difference
CruiseSpeed	60	forward speed

Raise the gain and it hugs the wall more tightly but starts to weave; lower it and it drifts wide on corners. These are the first numbers to play with.

Line following: three cases

The line follower carries two line sensors: one **centre** sensor ahead of the robot sitting over the line, and one **side** sensor ahead and out to the robot's **left**.

Putting the second sensor to one side rather than straddling the line is the whole trick. It makes two yes/no readings tell the robot which way it has drifted:

Centre	Side (left)	Meaning	Action
on	off	dead on the line	go straight
on	on	line sliding left	ease left
off	on	line has gone left	turn left
off	off	line went the <i>other</i> way	turn right

```
if( CentreOnLine )
{
    Result = SideOnLine ? -CorrectingTurn : 0.0f;
}
else
{
    Result = SideOnLine ? -SearchTurn : SearchTurn;
}
```

The last row is the clever one and it is what carries the robot around sharp corners. If **neither** sensor can see the line, the line cannot have gone left, because the left sensor would have caught it. So it must have gone right, and turning right sweeps it back into view. Two bits of information, and the robot can still recover from losing the line entirely.

`SimpleLine.map` is a seven-sided loop with one concave corner, so the robot has to turn both ways — a controller that only handled one direction would not get round.

8. `CWorld` — the room and the line

`CWorld` owns both loops and answers questions about them. It never reaches into a robot; robots ask it things.

`CLoopReader` reads exactly one loop per file, so the world holds two readers' worth of data: the walls and the line. They are kept apart deliberately, because they mean different things — a wall is sensed at a distance and collided with, a line is detected underfoot.

Three queries, one per job:

```
float DistanceToWallAlong( const Vec2D& arFrom, float aHeadingRadians ) const;
float DistanceToNearestWall( const Vec2D& arPoint ) const;
bool  IsOverLine( const Vec2D& arPoint ) const;
```

Each walks the loop's vertices as a series of segments. The loop is closed, so the last vertex joins back to the first; taking the next vertex as `(i + 1) % Count` covers that join without a special case:

```
for( std::size_t i = 0; i < Count; ++i )
{
    const Vec2D& rStart = mWallVertices[i];
    const Vec2D& rEnd   = mWallVertices[( i + 1 ) % Count];
    // ... test against this segment
}
```

`IsOverLine` treats the line as 5 units wide, as the handout specifies, so a point is over it when it is within 2.5 units of any of the line's segments.

9. `CGeometry` — the maths at the bottom

Every piece of trigonometry in the program is here, so that no other class contains any. All members are static; the class is never instantiated.

Ray against segment

Used by the range sensors: fire a ray from the sensor and find where it first crosses a wall.

A point on the ray is `Origin + t * Direction`, for `t >= 0`. A point on the segment is `Start + u * (End - Start)`, for `0 <= u <= 1`. They meet where those are equal. Writing **D** for the direction, **E** for `End - Start` and **F** for `Start - Origin`, the two component equations are:

$$\begin{aligned} t \cdot D.x - u \cdot E.x &= F.x \\ t \cdot D.y - u \cdot E.y &= F.y \end{aligned}$$

Two equations, two unknowns, solved by Cramer's rule. The ray hits when **t** is not behind the origin and **u** lands between the segment's ends. Because **D** is a unit vector, **t** is the distance travelled — which is exactly the sensor reading.

The determinant being zero means the ray and the segment are parallel, so they never meet. That is tested as `fabs(Determinant) > Epsilon` rather than `!= 0`, because floating point values are almost never exactly zero.

Point to segment

Used for line detection and for collisions: how far is this point from this segment.

Project the point onto the infinite line through the segment, then **clamp** the projection to the range `[0, 1]`. The clamp is what makes it a segment rather than an infinite line: a point off the end of a wall measures to the end of that wall, not to empty space where the wall would have been.

The rest

`UnitVector`, `PointAlong`, `Add`, `Subtract`, `Scale`, `Length`, `DistanceBetween` and `NormaliseAngle` are the small operations that keep trigonometry out of every other file. `NormaliseAngle` wraps a heading into `[0, 2*PI]` so that a robot turning in circles for a long run does not accumulate an ever-growing number.

10. Design decisions, and what you might change

These are the calls the draft makes. Each is defensible and each has a real alternative. They are worth arguing about between the two of you, because the argument is what A3 asks you to write up and what the 30 design marks are for.

1. Inheritance for the robots

As drafted: an abstract `CRobot` with everything shared, and two subclasses that differ only in `SenseAndSteer` and `DrawSensors`.

The alternative: one concrete `CRobot` class that *has* a controller and a set of sensors, chosen at construction. Composition instead of inheritance.

The trade-off: inheritance is simpler to read and matches “a wall follower *is* a robot”. Composition would let you swap a controller at run time and would avoid a class explosion if you later wanted four robot types with mixed sensors. For two robot types that never change mid-run, inheritance is the lighter answer — but you should be able to say why you chose it.

2. `CWheel` exists as a class

As drafted: a class holding a speed and a lateral offset.

The alternative: two floats in `CRobot`.

The trade-off: the handout explicitly specifies “two independently controlled wheels”, and having the wheels supply their own offsets makes the differential-drive maths read as a description of the mechanism rather than as bare arithmetic. Against that, it is a class with one mutable field, which some would call over-engineering. Genuinely arguable either way.

3. Sensors share a base for geometry, not for polymorphism

As drafted: `CSensor` holds the mounting arithmetic; `CRangeSensor` returns a float and `CLineSensor` returns a bool, so the robots hold concrete types.

The alternative: give `CSensor` a virtual `Read()` returning a normalised float, so a robot could hold `std::vector<CSensor*>` and not care what it has.

The trade-off: the draft is honest — the two readings really are different kinds of thing, and forcing “is the floor painted” into a float loses meaning. But the alternative demonstrates polymorphism more strongly, and polymorphism is a Week 2 topic the unit wants to see. A defensible middle: keep the typed readings but give `CSensor` a virtual `Draw()`, which the draft does not.

4. The `Color` leak

`CRender.h` claims the rest of the program never sees a raylib type, but it includes `raylib.h` and its drawing functions take raylib's `Color`. So the claim is not quite true, and it is a wart in the supplied code rather than in yours.

The draft confines it to `CPalette`, so there is exactly one file to change. You are explicitly allowed to modify the provided code, so **closing the leak properly** — defining your own colour type in `CRender.h` and converting inside `CRender.cpp` — would be a strong thing to have done and to write about. It is the kind of change §2.4 style thinking rewards: you noticed the abstraction was leaky and you fixed it.

5. Fixed-timestep integration

As drafted: turn, then move in a straight line, once per sixtieth of a second.

The alternative: integrate the arc properly, or use a smaller step.

The trade-off: at this step size the difference is invisible. Worth a sentence acknowledging it is an approximation rather than pretending it is exact.

6. Lap detection by distance from the start

As drafted: leave a radius, come back to a smaller one.

The alternative: track cumulative heading change (a full lap turns through 360 degrees), or count wall segments passed.

The trade-off: the drafted test is simple and readable but, as section 5 admits, it could be fooled. The cumulative-heading test is barely harder and is genuinely a lap test. This is probably the weakest part of the draft and the easiest place to make a real improvement of your own.

7. Things not yet done that the spec allows

- Collisions are counted and reported but the robot **passes through walls**. Nothing stops it. The hurdle only requires the count to be 10 or fewer, so this is legal, but making the robot physically stop or slide along the wall would be more realistic.
 - The controllers are pure proportional. Adding a derivative term would damp the weaving on long straights.
 - Neither robot slows down for corners.
-

11. What is still yours

The code covers A1 and A2 only. The rest of the lab is untouched, and most of the marks outside the code are in it.

Component	Marks	State
A0 Team Setup	10	Not started — needs both of you
Design Quality	30	Draft exists; the decisions are yours to make
Code Quality	30	Draft is commented to the unit's six-spot standard
A3 Post-Mortem	10	Not started — needs both of you
A4 ROS Tutorials	20	Not started — independent of all of this
A5 Noise Bonus	+10	Not started
Self-assessment	+5	Not started

Immediate blockers

1. **Join the Lab 2 Canvas group.** Lab 2 uses a different group set from Lab 1, and you are not in one yet. The submission is a single group submission.
2. **Fill in the file headers.** Every file says `Written by SID 540700701 and SID <PARTNER SID>` and `Practical section: <SECTION>`. Both placeholders need filling.

The SID attribution is deliberate. Lec 1B asks for your **name** in every file header, but the handout requires the report — which contains your code appendix — to carry SIDs only and no names. Attributing by SID satisfies both.

The report deliverables the code feeds

- **A UML class diagram** in the unit's simplified standard, the same one used for Task 3. Section 4's call tree and section 5's ownership are the raw material.
- **A screenshot** at the end of a run showing the completed trails.
- **Your console output**, showing any collisions.
- **The code appendix**, as text, not images. The handout gives the command:

```
find . -name "*.cpp" -o -name "*.h" | sort -V | xargs enscript -r --color=1 -C -  
Ecpp -fCourier8 -o - | ps2pdf - code.pdf  
pdfunite <report>.pdf code.pdf <combined>.pdf
```

The obvious split

A4 is completely independent of the simulator. ROS 2 Jazzy tutorials through Beginner: Client Libraries, plus augmenting the Pluginlib example with a House shape. One of you can do that start to finish while the other works on the simulator, then swap for review. It is 20 marks and it shares nothing with the C++ above.

12. Building and running

```
# macOS, raylib from Homebrew
g++ -Wall -Wextra *.cpp $(pkg-config --cflags --libs raylib) -o RobotSimulator

# Linux, raylib from apt - this is the command your marker uses
g++ -Wall -Wextra *.cpp -lraylib -o RobotSimulator

./RobotSimulator
```

Both `A1/` and `A2/` build with the same command and both are clean under `-Wall -Wextra`.

Before you submit, build it once on Linux with the plain command. The Minimum Functionality Hurdle requires your code to build that way, and a submission that does not clear the hurdle earns zero for A1 and A2 regardless of how good the design is. Your UTM virtual machine is the place to check.